



7. Trabalhando com listas em Python

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Links e mídias	Pasta "Aula 7"
 Última atualização	@17 de novembro de 2025 03:59
 Status	Done
 Data início	@08/11/2025
 Data conclusão	@09/11/2025 20:43
 Esforço	Médio

Anotações

Vídeo 1 - Introdução

Estrutura de dados:

Vídeo 2 - Listas: criação e acesso aos dados

Exemplos de listas

Acesso direto

Listas aninhadas

Fatiamento

Compreensão de listas

Vídeo 3 - Métodos da classe `list`

`list.append`
`list.clear`
`list.copy`
`list.count`
`list.extend`
`list.index`
`list.pop`
`list.remove`
`list.reverse`
`list.sort`
`len`
`sorted`

Exercícios

Arquivos

Anotações

Vídeo 1 - Introdução

Estrutura de dados:

- Listas
- Tuplas
- Conjuntos
- Dicionários
- Funções

Apresentação do repositório no Github com os arquivos usados no curso.

Vídeo 2 - Listas: criação e acesso aos dados

Listas podem armazenar qualquer tipo de objeto de maneira sequencial. Em Python, **tudo é um objeto**, portanto listas podem armazenar qualquer coisa: números, números de ponto flutuante, strings, dicionários, listas (conhecido como **listas aninhadas**), etc... E uma lista pode misturar valores, não precisa conter só um tipo de dado.

Para criar (declarar) listas, podemos usar o construtor `list` (também com a função `range`) ou colchetes (`[]`), separando os valores dentro deles por vírgulas (`,`).

Listas são **objetos mutáveis**, isto é, podemos alterar seus valores após a criação.

Exemplos de listas

```
frutas = ["laranja", "maca", "uva"]
```

```
frutas = [] # lista vazia
```

```
letras = list("python") # criação de lista a partir de uma string, exibe cada letra da palavra entre parênteses  
# ["p", "y", "t", "h", "o", "n"]
```

```
numeros = list(range(10)) # criação de lista a partir da função range, que exibe os números de 0 a 9  
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
carro = ["Ferrari", "F8", 4200000, 2020, 2900, "São Paulo", True, ["BMW", "Fiat", "Ford"]]
```

Acesso direto

A lista é uma sequência, assim podemos acessar seus dados com seus respectivos índices. Lembrando sempre que os índices em Python começam de `0`.

```
frutas = ["maçã", "laranja", "uva", "pera"]
```

```
print(frutas[0]) # maçã
print(frutas[2]) # uva
```

Sequências em Python proporcionam a indexação negativa, isto é, contando de trás para frente. A indexação, neste caso, começa em `-1`.

```
frutas = ["maçã", "laranja", "uva", "pera"]

print(frutas[-1]) # pera
print(frutas[-3]) # laranja
```

Listas aninhadas

Como as listas em Python podem armazenar outras listas, podemos criar estruturas bidimensionais (tabelas) e acessar seus dados informando os índices de linha e coluna.

```
matriz = [
    [1, "a", 2] # isto é uma linha, com 3 colunas
    ["b", 3, 4]
    [6, 5, "c"]
] # esta matriz possui 3 linhas e 3 colunas, uma matriz 3 x 3

matriz[0] # [1, "a", 2]
matriz[0][0] # 1
matriz[0][-1] # 2
matriz[-1][-1] # c
```

Fatiamento

Além de acessar elementos diretamente, podemos extrair um conjunto de valores de uma sequência. Para isso, basta passar o índice inicial e/ou final para acessar o conjunto. Podemos, também, informar quantas posições o cursor deve “pular” no acesso.

```
lista = ["p", "y", "t", "h", "o", "n"]

print(lista[2:]) # ["t", "h", "o", "n"], do índice 2 até o final
print(lista[:2]) # ["p", "y"], até o índice 1
```

```
print(lista[1:3]) # ["y", "t"], do índice 1 ao 2
print(lista[0:3:2]) # ["p", "t"], do índice 0 até o 2, contando de 2 em 2. O índice 0, aqui, poderia ser omitido.
print(lista[:]) # ["p", "y", "t", "h", "o", "n"], todos os índices
print(lista[::-1]) # ["n", "o", "h", "t", "y", "p"], todos os índices, de trás para frente
```

Compreensão de listas

A compreensão de lista oferece uma sintaxe mais curta para quando precisamos:

- criar uma nova lista com base nos valores de uma lista existente (filtros)
- gerar uma nova lista alterando os elementos de uma lista existente de alguma forma

```
# Filtrar lista
numeros = [1, 30, 21, 2, 9, 65, 34]
pares = [numero for numero in numeros if numero % 2 == 0]
print(pares)

# Modificar valores
numeros = [1, 30, 21, 2, 9, 65, 34]
quadrado = [numero**2 for numero in numeros]
print(quadrado)
```

Lembrando que a parte do código que define uma condicional (`if`) é opcional.

Vídeo 3 - Métodos da classe `list`

`list.append`

Adiciona elementos a uma lista.

```
lista = []

lista.append(1)
```

```
lista.append("Python")
lista.append([40, 30, 20])
```

`list.clear`

Usado para limpar (eliminar os elementos de) uma lista.

```
lista = [1, "Python", [40, 30, 20]]

print(lista) # [1, "Python", [40, 30, 20]]

lista.clear()

print(lista) # []
```

`list.copy`

Copia uma lista para que a original não seja alterada em outro ponto do código.

```
lista = [1, "Python", [40, 30, 20]]

l2 = lista.copy()

print(l2) # [1, "Python", [40, 30, 20]]
```

No exemplo acima, podemos alterar a `l2` e a lista original permanece intacta.

`list.count`

Conta a ocorrência dos elementos em uma lista.

```
cores = ["vermelho", "azul", "verde", "azul", "amarelo", "verde"]

print(cores.count("vermelho")) # 1
print(cores.count("azul")) # 2
print(cores.count("verde")) # 2
print(cores.count("amarelo")) # 1
```

`list.extend`

Une um ou mais elementos à lista, similar ao `append`.

```
linguagens = ["python", "js", "c"]

print(linguagens) # ["python", "js", "c"]

linguagens.extend(["java", "csharp"])

print(linguagens) # ["python", "js", "c", "java", "csharp"]
```

`[].index`

Exibe o número do índice do parâmetro colocado entre os parênteses.

```
linguagens = ["python", "js", "c", "java", "csharp"]

print(linguagens.index("java")) # 3
print(linguagens.index("python")) # 0
```

Lembrando que é exibida a primeira ocorrência do termo entre parênteses, então se há duas ocorrências de um termo, é exibido o índice da primeira. Combinando este método com `count`, podemos alterar todas as ocorrências de um termo, até que o `count` seja `0`.

`[].pop`

Exclui o último elemento da lista. Pode ser usado com um índice para excluir um elemento específico.

```
linguagens = ["python", "js", "c", "java", "csharp"]

print(linguagens.pop()) # csharp
print(linguagens.pop()) # java
print(linguagens.pop()) # c
print(linguagens.pop(0)) # python
print(linguagens) # ['js']
```

`[].remove`

Diferente de `pop`, em vez do índice, passa-se o objeto em si como parâmetro.

```
linguagens = ["python", "js", "c", "java", "csharp"]

linguagens.remove("c")

print(linguagens) # ["python", "js", "java", "csharp"]
```

Este método também exclui a primeira ocorrência encontrada.

[].reverse

Inverte a ordem da lista, exibindo-a de trás para frente.

```
linguagens = ["python", "js", "c", "java", "csharp"]

linguagens.reverse()

print(linguagens) # ["csharp", "java", "c", "js", "python"]
```

[].sort

Ordena uma lista.

- Sem parâmetros entre os parênteses (o padrão), ordena a lista alfabética ou numericamente de forma crescente.
- Aceita o parâmetro `reverse=True`, que inverte sua ordem alfabética ou numérica.
- Aceita funções como parâmetros.
- As funções podem ser combinadas com `reverse=True`.

```
linguagens = ["python", "js", "c", "java", "csharp"]
linguagens.sort() # ["c", "csharp", "java", "js", "python"]
print(linguagens)

linguagens = ["python", "js", "c", "java", "csharp"]
linguagens.sort(reverse=True) # ["python", "js", "java", "csharp", "c"]
print(linguagens)

linguagens = ["python", "js", "c", "java", "csharp"]
```

```
linguagens.sort(key=lambda x: len(x)) # ["c", "js", "java", "python", "csharp"], ordenando por comprimento da palavra
print(linguagens)
```

```
linguagens = ["python", "js", "c", "java", "csharp"]
linguagens.sort(key=lambda x: len(x), reverse=True) # ["python", "csharp", "java", "js", "c"]
# ordenando por comprimento da palavra
print(linguagens)
```

len

Esta função que acompanha a instalação exibe o comprimento de um elemento da lista ou de uma lista em si.

```
linguagens = ["python", "js", "c", "java", "csharp"]

print(len(linguagens)) # 5
print(len(linguagens[3])) # 4
```

sorted

Esta função que acompanha a instalação também serve para ordenar os elementos de uma lista:

- Sem parâmetros entre os parênteses (o padrão), ordena a lista alfabética ou numericamente de forma crescente.
- Aceita o parâmetro `reverse=True`, que inverte sua ordem alfabética ou numérica.
- Aceita funções como parâmetros.
- As funções podem ser combinadas com `reverse=True`.

```
linguagens = ["python", "js", "c", "java", "csharp"]

print(sorted(linguagens, key=lambda x: len(x))) # ["c", "js", "java", "python", "csharp"]
print(sorted(linguagens, key=lambda x: len(x), reverse=True)) # ["python", "csharp", "java", "js", "c"]
```

Exercícios

100% ⇒ 4/4 corretas

Arquivos

Pasta "Aula 7"